Chapter 1: Foundations of Transformers & Transformer Architecture

1. Introduction

Natural Language Processing (NLP) is the field of Artificial Intelligence that enables computers to understand and generate human language. Tasks such as translation, summarization, question answering, sentiment analysis, and chatbots all fall under NLP.

For many years, Recurrent Neural Networks (RNNs) and their improved variants (LSTMs and GRUs) were the dominant architectures for sequence modeling. They achieved impressive results but struggled with long sequences and slow training.

The Transformer architecture, introduced in 2017 in the paper **"Attention Is All You Need"**, completely changed this landscape. Instead of processing words sequentially, it processes all words simultaneously using an attention mechanism, enabling efficient parallel computation and better understanding of long-range dependencies.

Today, almost every modern Large Language Model (LLM), including GPT, BERT, Llama, Gemini, and many others, is built upon the Transformer architecture.

---

2. Why Sequence Models Were Needed

Traditional neural networks assume that all inputs are independent.

Consider the sentence:

> "The cat sat on the mat."

The meaning of the word **sat** depends on the word **cat**.

Similarly,

> "She moved to London because she got a new job."

The second occurrence of **she** refers to the same person mentioned earlier.

Language contains dependencies between words.

Traditional feed-forward neural networks cannot naturally model these sequential relationships.

Therefore sequence models such as RNNs were introduced.

---

3. Recurrent Neural Networks (RNNs)

An RNN processes one word at a time while maintaining a hidden state that carries information from previous words.

Example:

```
I → love → Deep → Learning
```

Processing flow:

```
Input1 → Hidden1

           ↓

Input2 → Hidden2

           ↓

Input3 → Hidden3

           ↓

Input4 → Hidden4
```

Each hidden state depends on the previous one.

The hidden state acts like memory.

It carries information from earlier words to later words.

---

4. Limitations of RNNs

Although RNNs introduced memory, they suffered from several problems.

4.1 Sequential Processing

RNNs cannot process all words simultaneously.

They must wait for the previous computation to finish.

```
Word1

↓

Word2

↓

Word3

↓

Word4
```

This prevents parallel computation and slows training significantly.

4.2 Vanishing Gradient Problem

During backpropagation, gradients become smaller and smaller as they travel through many time steps.

As a result, the network gradually forgets earlier information.

Example:

> "The boy who lives in Delhi and studies engineering at IIT won the scholarship."

The model may forget the subject **boy** by the time it reaches **won**.

4.3 Difficulty Learning Long-Range Dependencies

Suppose the sentence is:

> "The movie that we watched yesterday after returning from the long vacation was amazing."

The relationship between **movie** and **amazing** spans many intermediate words.

RNNs often struggle to preserve such long-distance information.

---

5. LSTM and GRU

Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) were introduced to solve the vanishing gradient problem.

They introduced gates that decide:

- What information to keep
- What information to forget
- What information to output

These mechanisms significantly improved long-term memory.

However, they still processed words sequentially.

```
Word1 → Word2 → Word3 → Word4 → Word5
```

The sequential bottleneck remained.

Training on very long sequences remained slow.

---

6. The Birth of the Transformer

Researchers asked a simple question:

> "Why should a model read one word at a time?"

Humans often understand a sentence by looking at multiple words simultaneously.

The solution was to remove recurrence entirely and replace it with an attention mechanism.

Instead of storing information in hidden states, every word directly communicates with every other word.

This idea led to the Transformer architecture.

The famous paper introducing this idea was titled:

> **Attention Is All You Need**

because attention replaced recurrence completely.

---

7. Main Idea Behind Transformers

The core idea is remarkably simple:

> Every word should decide which other words are important for understanding its meaning.

For example,

> "The animal didn't cross the road because it was tired."

The word **it** should attend strongly to **animal**, not **road**.

Attention allows the model to learn these relationships automatically.

---

8. High-Level Transformer Architecture

The original Transformer consists of two components:

- Encoder
- Decoder

```
Input Sentence

↓

Embedding

↓

Positional Encoding

↓

Encoder Stack

↓

Context Representation

↓

Decoder Stack
```

```
↓

Linear Layer

↓

Softmax

↓

Output Sentence
```

The encoder understands the input.

The decoder generates the output.

---

9. Encoder–Decoder Analogy

Imagine a student translating English into French.

Step 1:

The student first reads and understands the English sentence.

This corresponds to the **Encoder**.

Step 2:

The student then writes the French sentence using that understanding.

This corresponds to the **Decoder**.

Thus,

Encoder = Understanding

Decoder = Generation

---

10. Input Embeddings

Neural networks cannot process raw text.

Words must first be converted into numerical vectors.

Example:

```
Dog

↓

[0.12, 0.56, -0.31, ...]
```

This dense vector representation is called an **Embedding**.

Words with similar meanings often have similar embeddings.

For example,

```
King

Queen

Prince

Princess
```

are located close together in embedding space.

Embeddings capture semantic relationships between words.

---

11. Why Positional Encoding is Needed

Unlike RNNs, Transformers process all tokens simultaneously.

Suppose we have two sentences:

```
Dog bites man

Man bites dog
```

Both contain identical words.

Without position information, the Transformer would treat them as identical.

Word order would be lost.

Therefore position information must be explicitly added.

---

12. Positional Encoding

The embedding of each word is combined with a positional vector.

```
Final Input

=

Embedding

+

Position Encoding
```

Now every token contains:

- semantic meaning
- positional information

The model knows both **what the word is** and **where it appears**.

---

13. Intuition Behind Positional Encoding

Imagine every student in a classroom wears a numbered badge.

```
Student A → Seat 1

Student B → Seat 2

Student C → Seat 3
```

Even if students change clothes, their seat numbers identify their positions.

Similarly, positional encoding tells the Transformer where each word occurs in the sequence.

---

14. Data Flow Through the Transformer

The complete data pipeline is:

```
Input Sentence

↓

Tokenization

↓

Embeddings

↓

Positional Encoding

↓

Encoder Layers

↓

Context Representation

↓

Decoder Layers

↓

Linear Layer

↓

Softmax

↓

Generated Output
```

Each stage transforms the representation into richer contextual information.

---

15. Inside the Encoder (Preview)

Each encoder layer contains:

```
Input

↓

Multi-Head Attention

↓

Add & LayerNorm

↓

Feed Forward Network

↓

Add & LayerNorm

↓

Output
```

This structure is repeated multiple times.

Each repetition improves contextual understanding.

---

16. Inside the Decoder (Preview)

Each decoder layer contains:

```
Input

↓

Masked Self Attention

↓
```

```
Add & LayerNorm

↓

Cross Attention

↓

Add & LayerNorm

↓

Feed Forward Network

↓

Add & LayerNorm

↓

Output
```

The masking operation prevents future information leakage.

Cross-attention allows the decoder to use encoder information.

---

17. Why Transformers Revolutionized AI

Transformers solved many problems of RNNs:

✓ Parallel computation

✓ Better long-range dependency learning

✓ Faster GPU utilization

✓ Better contextual understanding

✓ Massive scalability

These advantages enabled models with billions of parameters and transformed NLP research.

---

18. Chapter Summary

• Traditional neural networks cannot model sequential language naturally.

• RNNs introduced memory but suffered from sequential computation and vanishing gradients.

• LSTMs and GRUs improved memory but remained sequential.

• Transformers removed recurrence entirely and replaced it with attention.

• The architecture consists of an Encoder and a Decoder.

• Words are converted into embeddings before processing.

• Positional encoding provides order information.

• The encoder builds contextual representations.

• The decoder generates output tokens using those representations.

This chapter provides the conceptual foundation needed to understand the attention mechanism, which will be studied in the next chapter.

Interview Questions

Q1. Why were Transformers introduced?

To overcome the sequential computation and long-range dependency problems of RNNs and LSTMs.

Q2. Why can't Transformers understand word order naturally?

Because they process all tokens simultaneously instead of sequentially.

Q3. What is the role of positional encoding?

It provides explicit position information to each token.

Q4. What are the two main components of the original Transformer?

Encoder and Decoder.

Q5. Why are Transformers faster than RNNs?

Because all tokens can be processed in parallel rather than sequentially.

Chapter 2: Self-Attention and Query-Key-Value (QKV)

1. Introduction

The Transformer architecture is built around a single revolutionary idea:

> **Instead of processing words one by one, every word should be able to directly examine every other word and decide which ones are important for understanding its meaning.**

This mechanism is called **Self-Attention**.

Self-attention is the heart of the Transformer architecture. It enables every token in a sentence to gather contextual information from all other tokens, allowing the model to understand relationships regardless of their distance.

Unlike RNNs, information does not need to travel through many hidden states. Instead, every token communicates directly with every other token in a single operation.

---

2. What is Attention?

Humans naturally focus on important information.

Consider the sentence:

> **"The animal didn't cross the road because it was tired."**

When you read the word **it**, your brain immediately associates it with **animal**, not **road**.

Your brain automatically assigns more importance to the relevant word.

This selective focus is called **attention**.

In deep learning,

Attention is a mechanism that allows the model to assign different importance (weights) to different words while processing a sentence.

Instead of treating every word equally, the model learns which words are more useful for understanding the current token.

---

3. Self-Attention

Attention can operate between different sequences or within the same sequence.

When a word attends to words in the same sentence, it is called **Self-Attention**.

Example:

```
The cat sat on the mat.
```

While processing the word **sat**, the model may pay attention to:

- cat
- on
- mat

because these words provide context.

Every word communicates with every other word.

```
The ↔ cat ↔ sat ↔ on ↔ the ↔ mat
```

This interaction creates contextual representations.

---

4. Why Self-Attention is Better than RNN Memory

Consider the sentence:

> **The boy who lives in Delhi and studies engineering at IIT won the scholarship.**

The word **won** depends on **boy**.

In an RNN, this information must pass through many hidden states.

```
boy → who → lives → in → Delhi → and → studies → engineering → at → IIT → won
```

Information gradually weakens.

In Self-Attention,



The connection is direct.

Long-range dependencies become much easier to learn.

---

5. The Query-Key-Value (QKV) Concept

Every input embedding is transformed into three different vectors:

- Query (Q)
- Key (K)
- Value (V)

These vectors are generated by multiplying the embedding with three learnable weight matrices.

Mathematically,

$$Q = XW_Q$$

$$K = XW_K$$

$$V = XW_V$$

where

- X = input embedding matrix
- W_Q = Query weight matrix
- W_K = Key weight matrix
- W_V = Value weight matrix

These matrices are learned during training.

---

6. Intuition Behind QKV

Think of searching for a book in a library.

Your question:

> "I need a book about Deep Learning."

This is the **Query**.

Each book has a title.

That title acts as the **Key**.

Inside each book is the actual information.

That information is the **Value**.

Process:

```
Question (Query)

↓

Compare with Titles (Keys)

↓

Find Best Matches

↓

Read Information (Values)

↓

Generate Answer
```

This is exactly how Transformer attention operates.

---

7. Classroom Analogy

Imagine a classroom.

Each student asks:

> "Who among my classmates has information relevant to me?"

The question is the Query.

Each student wears an identity badge (Key).

The knowledge they possess is the Value.

Students compare their queries against everyone's keys.

The most relevant students contribute their knowledge.

Self-attention behaves similarly.

---

8. Generating Q, K and V

Suppose the sentence is

```
Riya loves AI
```

Each embedding generates:

```
Riya

↓

Q₁ K₁ V₁

--------------------

loves

↓

Q₂ K₂ V₂

--------------------

AI

↓

Q₃ K₃ V₃
```

Now each Query will compare itself against every Key.

---

9. Dot Product Attention

Suppose we process the word **loves**.

Its Query compares with

- Key(Riya)
- Key(loves)
- Key(AI)

using the dot product.

The similarity score is

$$Score = Q \cdot K$$

A large score indicates a strong relationship.

A small score indicates a weak relationship.

This comparison is repeated for every token.

---

10. Score (Correlation) Matrix

Instead of comparing one pair at a time, the Transformer computes all similarities simultaneously.

Example:

```
              Keys

          Riya  Loves  AI

Queries

Riya        8     3     2

Loves       2     7     5

AI          1     3     9
```

Rows represent Queries.

Columns represent Keys.

Each element measures similarity.

This matrix is called the **Score Matrix** or **Correlation Matrix**.

---

11. Matrix Representation

Rather than computing every similarity individually,

Transformers compute

$$QK^T$$

where

- Q contains all Query vectors
- K contains all Key vectors

This produces the complete score matrix in one matrix multiplication.

If there are N tokens,

the matrix size becomes

$$N \times N$$

because every word compares with every other word.

---

12. Why Scaling is Necessary

Suppose vector dimension becomes very large.

Dot products also become large.

Large numbers cause Softmax to produce extremely sharp probabilities, making gradients unstable.

Therefore attention scores are divided by

$$\sqrt{d_k}$$

where

d_k

is the dimension of the Key vector.

The scaled attention score becomes

$$\frac{QK^T}{\sqrt{d_k}}$$

Scaling stabilizes training.

---

13. Softmax

The score matrix contains raw similarity scores.

Example:

```
[2 5 3]
```

These values are not probabilities.

Softmax converts them into probabilities.

Example:

```
Raw Scores

[2 5 3]

↓

Softmax

[0.04 0.84 0.12]
```

Properties:

- Values lie between 0 and 1.
- Sum of probabilities equals 1.

These probabilities become **attention weights**.

---

14. Weighted Sum of Value Vectors

Suppose

Attention Weights

```
[0.04 0.84 0.12]
```

Value vectors:

```
V₁

V₂

V₃
```

The output becomes

$$0.04V_1 + 0.84V_2 + 0.12V_3$$

Thus information from different words is combined according to their importance.

The resulting vector becomes the new contextual representation.

---

15. Numerical Example

Sentence:

```
Riya loves AI
```

Suppose attention scores for **loves** are

```
[2,5,3]
```

Assume

$$\sqrt{d_k} = 2$$

Scaled scores:

```
[1,2.5,1.5]
```

Applying Softmax gives

```
[0.14,0.63,0.23]
```

Suppose

```
Value(Riya)=10

Value(Loves)=20

Value(AI)=30
```

Output:

$$0.14 \times 10 + 0.63 \times 20 + 0.23 \times 30$$

$$= 1.4 + 12.6 + 6.9$$

$$= 20.9$$

The representation of **loves** now contains information from **Riya** and **AI** as well.

This is called a **contextual embedding**.

---

16. Complete Attention Formula

The complete attention mechanism is summarized as

$$Attention(Q, K, V) = Softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

The process consists of six steps:

1. Generate Query vectors.
2. Generate Key vectors.
3. Generate Value vectors.
4. Compute similarity scores.
5. Scale and normalize using Softmax.
6. Compute weighted sum of Value vectors.

The output is the contextual representation.

17. Complete Self-Attention Pipeline

```
Input Embeddings

↓

Generate Q K V

↓

QKᵀ

↓

Score Matrix

↓

Scale by √dk

↓

Softmax

↓

Attention Weights

↓

Multiply by Values

↓

Contextual Embeddings
```

Every token performs this computation simultaneously.

This parallel computation is one reason why Transformers are so efficient.

18. Why Self-Attention is Powerful

Self-attention provides several advantages:

- Every token directly communicates with every other token.
- Long-range dependencies are learned easily.
- Parallel computation becomes possible.
- Context is incorporated into every embedding.
- Rich semantic representations are produced.

This mechanism forms the foundation of every modern Large Language Model.

---

19. Chapter Summary

- Attention allows the model to focus on important words.
- Self-attention operates within the same sequence.
- Every embedding generates Query, Key and Value vectors.
- Query vectors compare with Key vectors using dot products.
- The resulting similarity matrix is called the Score Matrix.
- Scores are divided by $\sqrt{d_k}$ for numerical stability.
- Softmax converts scores into probabilities.
- These probabilities weight the Value vectors.
- The weighted combination produces contextual embeddings.
- Self-attention replaces recurrence and forms the core operation of the Transformer.

Interview Questions

Q1. What is Self-Attention?

A mechanism where each token attends to all other tokens in the same sequence to gather contextual information.

Q2. Why are Query, Key and Value needed?

They enable each token to search for relevant information and combine useful contextual information from other tokens.

Q3. Why do we divide by $\sqrt{d_k}$?

To prevent large dot products from making Softmax unstable and gradients too small.

Q4. What does the Score Matrix represent?

It contains similarity scores between every Query and every Key.

Q5. What is the final output of Self-Attention?

A contextual embedding where every token contains information gathered from the entire sequence.

Chapter 3: Multi-Head Attention and Encoder Internals

1. Introduction

In the previous chapter, we learned how **Self-Attention** allows every word in a sentence to communicate with every other word.

However, Self-Attention alone is not enough.

A sentence contains multiple types of relationships simultaneously:

- Grammatical relationships
- Semantic relationships
- Subject-verb relationships
- Location relationships
- Time relationships
- Object relationships

A single attention mechanism may capture only one dominant relationship and miss others.

To overcome this limitation, the Transformer introduces **Multi-Head Attention (MHA)**.

Instead of learning one relationship, the model learns **multiple relationships simultaneously** by using several attention heads in parallel.

---

2. Why One Attention Head is Not Enough

Consider the sentence:

> **"Riya moved to London because she got a new job."**

Suppose we are processing the word **she**.

There are multiple useful relationships:

- **Riya** → subject
- **London** → location
- **moved** → action
- **job** → reason

If there is only one attention head, it must learn all these relationships simultaneously.

It may focus strongly on **Riya** but ignore **job** or **London**.

This limits the richness of learned representations.

---

3. The Idea Behind Multi-Head Attention

Instead of using one attention mechanism, Transformers use several attention mechanisms simultaneously.

Each attention mechanism is called a **Head**.

Each head learns different patterns independently.

The outputs are then combined together.

---

4. Teacher Analogy

Imagine ten teachers grading the same essay.

Teacher 1 focuses on:

- Grammar

Teacher 2 focuses on:

- Vocabulary

Teacher 3 focuses on:

- Creativity

Teacher 4 focuses on:

- Logic

Teacher 5 focuses on:

- Presentation

Every teacher produces a different evaluation.

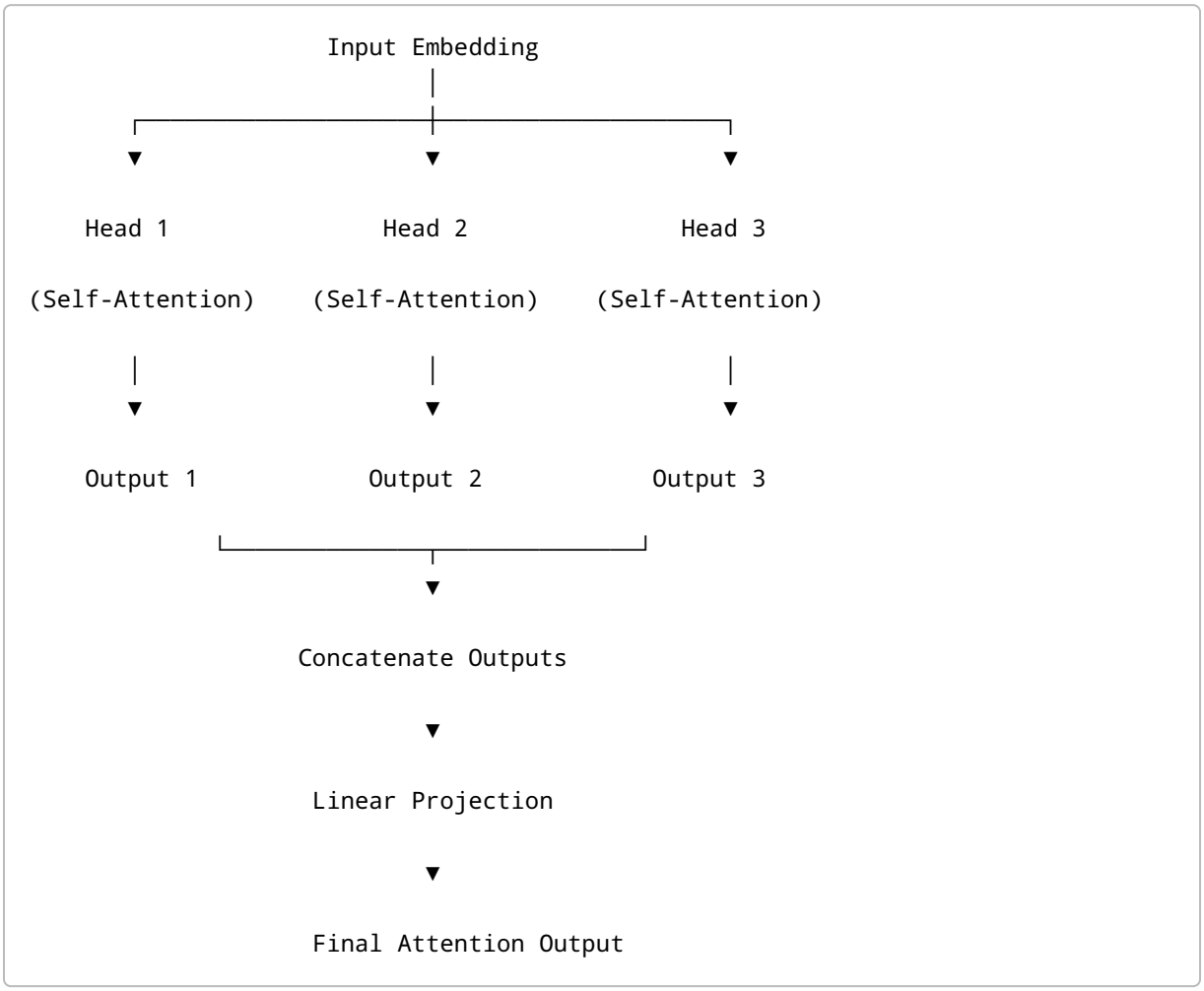
Finally all evaluations are combined.

The final assessment becomes much better than relying on a single teacher.

Multi-Head Attention works in exactly the same way.

---

5. Multi-Head Attention Architecture



Every head performs Self-Attention independently.

---

6. Separate Q, K and V for Every Head

Suppose there are **8 heads**.

The input embedding generates:

$$Q_1, K_1, V_1$$

$$Q_2, K_2, V_2$$

$$Q_3, K_3, V_3$$

...

$$Q_8, K_8, V_8$$

Each head has its own learnable projection matrices.

Therefore each head learns different relationships.

---

7. Example of Multiple Heads

Sentence:

> **"The boy who lives in Delhi studies at IIT."**

Suppose we process **studies**.

Head 1 may attend to:

```
boy
```

(subject)

Head 2 may attend to:

```
Delhi
```

(location)

Head 3 may attend to:

```
IIT
```

(institution)

Head 4 may attend to:

```
who lives
```

(relative clause)

Every head extracts different information.

---

8. Parallel Computation

One major advantage of Multi-Head Attention is parallel computation.

All heads operate simultaneously.

```
Input

↓

Head1

Head2

Head3

Head4

↓

Outputs Combined
```

Modern GPUs compute all heads in parallel.

This makes Transformers extremely efficient.

---

9. Concatenation

Suppose three heads produce:

```
Head1 = [2 4]

Head2 = [6 3]
```

```
Head3 = [1 8]
```

Concatenation joins them:

```
[2 4 | 6 3 | 1 8]

↓

[2 4 6 3 1 8]
```

The combined vector now contains information from all heads.

---

10. Linear Projection

After concatenation, the vector dimension becomes large.

A final learned linear transformation combines information from different heads.

```
Concatenated Output

↓

Linear Layer

↓

Final Attention Output
```

This allows different heads to interact with one another.

---

11. Complete Multi-Head Attention Formula

If there are h attention heads,

then

$$head_i = Attention(Q_i, K_i, V_i)$$

The outputs are concatenated:

$$Concat(head_1, head_2, \ldots, head_h)$$

Finally,

$$MultiHead = Concat(heads)W_O$$

where W_O is another learned projection matrix.

---

12. Residual Connections

Deep neural networks become difficult to train as layers increase.

Information may gradually disappear.

To preserve information, Transformers use **Residual Connections**.

Instead of replacing the input,

the model adds it back.

```
Input

↓

Attention

↓

Output

+

Original Input

↓

Final Output
```

Mathematically,

$$Y = X + Attention(X)$$

---

13. Why Residual Connections Help

Suppose attention accidentally loses useful information.

Residual connections ensure the original representation is preserved.

Think of it as:

```
Old Knowledge

+

New Knowledge

↓

Better Knowledge
```

Residual learning also improves gradient flow.

Training becomes easier.

---

14. Layer Normalization

Neurons often produce outputs with different scales.

Example:

```
100

2

0.5
```

Large differences make optimization unstable.

Layer Normalization rescales activations.

Example:

```
Before

100 2 0.5

↓

LayerNorm

↓

0.9 -0.4 -0.5
```

Values become centered and normalized.

Training becomes stable.

---

15. Why Layer Normalization is Important

Without normalization:

- exploding activations
- unstable gradients
- slower convergence

With normalization:

- stable optimization
- faster training
- improved accuracy

Every Transformer layer uses LayerNorm.

---

16. Feed Forward Network (FFN)

After attention, each token independently passes through a small neural network.

Architecture:

```
Input

↓
```

```
Linear Layer

↓

Activation (ReLU/GELU)

↓

Linear Layer

↓

Output
```

Unlike attention,

FFN processes every token separately.

---

17. Intuition Behind FFN

Attention gathers information.

FFN processes that information.

Think of attention as collecting ingredients.

FFN cooks the meal.

Both operations are necessary.

---

18. Complete Encoder Layer

One encoder layer consists of:

```
Input

↓

Multi-Head Attention
```

```
↓

Add & LayerNorm

↓

Feed Forward Network

↓

Add & LayerNorm

↓

Output
```

This is the basic building block of the encoder.

---

19. Encoder Data Flow

```
Input Embedding

↓

Multi-Head Attention

↓

Residual Connection

↓

LayerNorm

↓

Feed Forward Network

↓

Residual Connection

↓
```

```
LayerNorm

↓

Output Embedding
```

The output becomes input to the next encoder layer.

---

20. Why Multiple Encoder Layers are Stacked

One encoder layer captures simple relationships.

Multiple layers progressively refine representations.

Example:

Layer 1:

```
Dog

Animal
```

Layer 2:

```
Dog chases ball
```

Layer 3:

```
Dog is subject
```

Layer 4:

```
Entire sentence meaning
```

Higher layers capture increasingly abstract information.

---

21. Reading Analogy

Imagine reading a paragraph.

First reading:

Understand words.

Second reading:

Understand sentences.

Third reading:

Understand overall meaning.

Stacked encoder layers behave similarly.

Every layer refines understanding.

---

22. Complete Encoder Architecture

```
Embedding + Position

↓

Encoder Layer 1

↓

Encoder Layer 2

↓

Encoder Layer 3

↓

...

↓

Encoder Layer N
```

```
↓

Context Representation
```

The final output contains contextual embeddings for every token.

These embeddings are passed to the decoder.

---

23. Why Multi-Head Attention Makes Transformers Powerful

Instead of learning a single relationship,

the Transformer simultaneously learns:

- Syntax
- Grammar
- Semantics
- Subject relationships
- Object relationships
- Location information
- Time information
- Context

This dramatically improves language understanding.

---

24. Chapter Summary

- A single attention head cannot capture every relationship.
- Multi-Head Attention uses multiple attention mechanisms simultaneously.
- Every head learns different linguistic patterns.
- Outputs from all heads are concatenated.
- A linear projection combines all heads.
- Residual connections preserve information and improve gradient flow.
- Layer Normalization stabilizes optimization.
- Feed Forward Networks refine token representations.
- One encoder layer consists of Multi-Head Attention, Add & Norm, FFN, and Add & Norm.
- Multiple encoder layers gradually build deeper contextual understanding.

Interview Questions

Q1. Why is Multi-Head Attention better than single-head attention?

Because different heads learn different relationships simultaneously, producing richer contextual representations.

Q2. Why are residual connections used?

To preserve information and improve gradient flow during deep network training.

Q3. Why is Layer Normalization required?

It stabilizes activations and accelerates convergence during optimization.

Q4. What is the purpose of the Feed Forward Network?

It independently transforms each token's representation after attention has gathered contextual information.

Q5. What are the components of one encoder layer?

- Multi-Head Attention
- Residual Connection
- Layer Normalization
- Feed Forward Network
- Residual Connection
- Layer Normalization

Key Takeaway

The **Encoder** can be viewed as a **team of specialists**:

- **Multi-Head Attention** gathers information from multiple perspectives.
- **Residual Connections** preserve existing knowledge.
- **Layer Normalization** stabilizes learning.
- **Feed Forward Networks** refine representations.
- **Stacking multiple encoder layers** gradually builds a deep semantic understanding of the entire sentence.

This combination is what makes the Transformer encoder one of the most powerful sequence representation models ever developed.

Chapter 4: Decoder, Cross-Attention and Masking

1. Introduction

In the previous chapter, we learned how the **Encoder** converts an input sentence into rich contextual representations using Multi-Head Attention.

However, understanding the input is only half of the task.

The model must also **generate an output sequence**.

This responsibility belongs to the **Decoder**.

The Decoder generates one token at a time while continuously using the information learned by the Encoder.

Unlike the Encoder, the Decoder must ensure that it **never looks into the future** while generating text.

To achieve this, the Transformer introduces **Masked Self-Attention**.

---

2. What is the Decoder?

The Decoder is responsible for generating the output sequence.

Suppose we want to translate:

> **English:** I love AI

into

> **French:** J'aime l'IA

The Encoder first understands the English sentence.

The Decoder then generates the French sentence one word at a time.

```
Encoder
"I love AI"

↓
```

```
Context Representation

↓

Decoder

↓

J'

↓

aime

↓

l'IA
```

The Decoder acts like a writer that generates the output based on the Encoder's understanding.

---

3. Encoder vs Decoder

Think of a student during an examination.

The student first reads and understands the question paper.

This is the **Encoder**.

The student then writes the answer.

This is the **Decoder**.

Therefore,

Encoder = Understanding

Decoder = Generation

---

4. Decoder Architecture

Each Decoder layer contains three major components:

```
Input

↓

Masked Multi-Head Attention

↓

Add & LayerNorm

↓

Cross Attention

↓

Add & LayerNorm

↓

Feed Forward Network

↓

Add & LayerNorm

↓

Output
```

Unlike the Encoder, the Decoder has **two attention mechanisms**:

1. Masked Self-Attention
2. Cross-Attention

---

5. Why Masked Self-Attention?

Suppose the model is generating

> **I love deep learning**

While predicting **deep**, the model should only know:

```
I

love
```

It must **not know**

```
learning
```

because that word belongs to the future.

Otherwise the model would simply copy the answer instead of learning to predict it.

---

6. Why Future Information Must Be Hidden

Suppose training data is

```
I love deep learning
```

If the model can already see

```
deep learning
```

while predicting **deep**, prediction becomes trivial.

The model would cheat.

It would not actually learn language modeling.

Therefore future tokens must be hidden.

---

7. What is Masking?

Masking means:

> **Hide future tokens while computing attention.**

The Decoder can only attend to:

- current token
- previous tokens

It cannot attend to future tokens.

---

8. Example of Masking

Suppose the sequence is

```
I love AI very much
```

While predicting **AI**

Visible:

```
I

love
```

Hidden:

```
AI

very

much
```

The model predicts AI using only previous information.

---

9. Causal Mask Matrix

Suppose sentence length is four.

Without masking:

```
          I love AI today
```

```
I          ✓ ✓ ✓ ✓

love       ✓ ✓ ✓ ✓

AI         ✓ ✓ ✓ ✓

today      ✓ ✓ ✓ ✓
```

Every token sees every token.

After masking:

```
           I love AI today

I          ✓ ✗ ✗ ✗

love       ✓ ✓ ✗ ✗

AI         ✓ ✓ ✓ ✗

today      ✓ ✓ ✓ ✓
```

Future positions become invisible.

This is called the **Causal Mask**.

---

10. Mathematical View of Masking

Suppose attention scores are

```
[2 5 3 1]
```

Future positions are replaced by

```
[2 5 -∞ -∞]
```

Softmax computes

$$e^x$$

Since

$$e^{-\infty} = 0$$

future tokens receive zero probability.

Therefore they cannot influence prediction.

---

11. Why -∞ is Used

Suppose instead we used

```
[2 5 0 0]
```

Softmax would still assign small probabilities to future positions.

Using

$$-\infty$$

ensures

$$Softmax(-\infty) = 0$$

making future attention impossible.

---

12. Self-Attention vs Masked Self-Attention

Encoder:

```
Every token

↓

Can attend everywhere
```

Decoder:

```
Every token
```

```
↓

Can attend only left side
```

The Decoder never sees future words.

---

13. Cross-Attention

After Masked Self-Attention, the Decoder performs another attention operation.

This is called **Cross-Attention**.

Cross-Attention allows the Decoder to use Encoder information.

---

14. Intuition Behind Cross-Attention

Imagine translating:

> I love AI

When generating

```
aime
```

the Decoder should attend to

```
love
```

from the Encoder.

When generating

```
l'IA
```

the Decoder should attend to

```
AI
```

Thus Decoder continuously consults Encoder memory.

---

15. QKV in Cross-Attention

Self-Attention:

```
Q ← Decoder

K ← Decoder

V ← Decoder
```

Cross-Attention:

```
Query

↓

Decoder

Key

↓

Encoder

Value

↓

Encoder
```

The Decoder asks questions.

The Encoder provides answers.

---

16. Complete Decoder Layer

```
Input
```

```
↓

Masked Multi-Head Attention

↓

Residual Connection

↓

LayerNorm

↓

Cross Attention

↓

Residual Connection

↓

LayerNorm

↓

Feed Forward Network

↓

Residual Connection

↓

LayerNorm

↓

Output
```

This structure repeats multiple times.

---

17. Teacher Forcing During Training

Suppose sentence:

```
I love AI
```

Training pairs become

Input:

```
I
```

Target:

```
love
```

Next,

Input:

```
I love
```

Target:

```
AI
```

The model learns next-token prediction.

Masking prevents cheating.

This technique is called **Teacher Forcing**.

---

18. Inference (Text Generation)

Suppose prompt is

```
I love
```

Decoder predicts

```
AI
```

Append:

```
I love AI
```

Predict again:

```
because
```

Append:

```
I love AI because
```

Repeat until END token.

This is called **Autoregressive Generation**.

---

19. Complete Decoder Workflow

```
Previous Tokens

↓

Embedding

↓

Positional Encoding

↓

Masked Self Attention

↓

Cross Attention

↓
```

```
Feed Forward

↓

Linear Layer

↓

Softmax

↓

Next Token
```

The predicted token is appended to the input and the process repeats.

---

20. Complete Transformer Pipeline

```
Input Sentence

↓

Embedding

↓

Positional Encoding

↓

Encoder Stack

↓

Encoder Memory

↓

Decoder Input

↓

Masked Attention
```

```
↓

Cross Attention

↓

Feed Forward

↓

Linear Layer

↓

Softmax

↓

Output Token

↓

Repeat
```

This is the complete architecture proposed in the original Transformer paper.

---

21. Why Masking is the Foundation of GPT

GPT does not contain an Encoder.

It consists only of Decoder blocks.

The Decoder predicts one token at a time.

Masking ensures that every prediction depends only on previous tokens.

Without masking,

GPT could see future words and would fail as a language model.

Masking enables **autoregressive language modeling**, which powers ChatGPT and modern LLMs.

---

22. Chapter Summary

- The Decoder generates output sequences.
- Each Decoder layer contains Masked Self-Attention, Cross-Attention, and Feed Forward Networks.
- Masking prevents future information leakage.
- Future positions receive zero attention after Softmax.
- Cross-Attention allows the Decoder to access Encoder representations.
- Teacher Forcing speeds up training while masking prevents cheating.
- During inference, tokens are generated one at a time.
- Autoregressive generation is the basis of GPT and modern language models.

Interview Questions

Q1. Why does the Decoder use masking?

To prevent future information leakage during next-token prediction.

Q2. What is Cross-Attention?

Cross-Attention is an attention mechanism where Queries come from the Decoder while Keys and Values come from the Encoder output.

Q3. Why is the Decoder autoregressive?

Because each new token is generated using only previously generated tokens.

Q4. Why is $-\infty$ used in masking?

Because Softmax converts $e^{-\infty}$ to zero, completely blocking future tokens.

Q5. What is the difference between Encoder Self-Attention and Decoder Self-Attention?

Encoder Self-Attention allows every token to attend to every other token, while Decoder Self-Attention uses causal masking so tokens can attend only to themselves and previous tokens.

Key Takeaway

Think of the **Decoder** as a **writer with a memory**:

- **Masked Self-Attention** lets it read only what has already been written.
- **Cross-Attention** lets it consult the Encoder's understanding of the input sentence.
- **Feed Forward Networks** refine the generated representation.
- By repeating this process token by token, the Decoder produces a coherent output sequence.

This autoregressive decoding mechanism is the fundamental idea behind **GPT, ChatGPT, and nearly all modern generative language models.**

Chapter 5: GPT and Modern Large Language Models (LLMs)

1. Introduction

The original Transformer architecture consists of two components:

- Encoder
- Decoder

It was primarily designed for **sequence-to-sequence tasks**, such as machine translation.

However, modern language models like **GPT** perform a different task.

Instead of translating one sequence into another, GPT simply learns:

> **"Given the previous tokens, predict the next token."**

This simple objective allows GPT to generate essays, answer questions, write code, summarize documents, and perform hundreds of NLP tasks.

The success of GPT comes from combining the Transformer Decoder with massive datasets and large-scale training.

---

2. What is GPT?

GPT stands for

Generative Pre-trained Transformer

Each word has an important meaning:

Generative

It generates new text rather than only classifying existing text.

Pre-trained

It is first trained on enormous amounts of internet text before being fine-tuned for downstream tasks.

Transformer

Its architecture is based on the Transformer Decoder introduced in "Attention Is All You Need."

---

3. Why GPT Uses Only the Decoder

The original Transformer uses

```
Encoder

↓

Decoder
```

The Encoder understands the input sequence.

The Decoder generates another sequence.

Example:

English

↓

French

Translation requires both Encoder and Decoder.

GPT performs a different task.

Example:

```
The capital of France is _____
```

GPT simply predicts

```
Paris
```

There is no separate input and output sequence.

The prompt itself acts as the input.

Therefore the Encoder becomes unnecessary.

GPT uses only stacked Decoder blocks.

---

4. GPT Architecture

The architecture becomes

```
Input Prompt

↓

Tokenization

↓

Embedding

↓

Positional Encoding

↓

Decoder Layer 1

↓

Decoder Layer 2

↓

Decoder Layer 3

↓

...

↓

Decoder Layer N
```

```
↓

Linear Layer

↓

Softmax

↓

Next Token
```

Notice that the Encoder is completely removed.

---

5. Prompt Processing

Suppose the prompt is

```
The capital of France is
```

The tokenizer converts it into tokens:

```
The

capital

of

France

is
```

Each token is converted into embeddings.

Positional encoding is added.

The Decoder processes the sequence using Masked Self-Attention.

Finally Softmax predicts the most likely next token.

Example:

```
Paris
```

---

6. Autoregressive Language Modeling

GPT predicts one token at a time.

Suppose the prompt is

```
Artificial Intelligence is
```

Prediction:

```
transforming
```

Append:

```
Artificial Intelligence is transforming
```

Predict again:

```
the
```

Append:

```
Artificial Intelligence is transforming the
```

Predict again:

```
world
```

Continue until an END token is generated.

This process is called

Autoregressive Generation

because the model generates the sequence using its own previous outputs.

---

7. Token-by-Token Generation

Generation loop:

```
Prompt

↓

Predict Next Token

↓

Append Token

↓

Run Decoder Again

↓

Predict Next Token

↓

Append

↓

Repeat
```

Every generated token becomes part of the next input.

This iterative process continues until generation stops.

---

8. Why Masking is Essential

Suppose GPT is generating

```
The capital of France is Paris
```

While predicting

```
Paris
```

the model must only see

```
The

capital

of

France

is
```

It cannot see

```
Paris
```

Otherwise prediction would be trivial.

Masking prevents future information leakage and forces genuine learning.

---

9. Decoder Self-Attention in GPT

Suppose the prompt is

```
The boy lives in Delhi
```

When processing

```
Delhi
```

attention may focus on

- boy
- lives
- in

The Decoder uses previous context to build contextual embeddings before predicting the next token.

---

10. The Computational Problem

Suppose the prompt length is

100 tokens.

Predicting token 101 requires attention over all previous tokens.

Predicting token 102 again requires attention over all previous tokens.

The same Keys and Values would be recomputed repeatedly.

This becomes computationally expensive.

---

11. KV Cache

To solve this problem, GPT uses **Key-Value Cache (KV Cache).**

Instead of recomputing old Keys and Values every generation step, they are stored.

Example:

```
Token1

↓

Store K,V

Token2

↓

Store K,V
```

```
Token3

↓

Store K,V

New Token

↓

Compute only new Q,K,V
```

Previously computed Keys and Values are reused.

Only the newest token requires fresh computation.

---

12. Why KV Cache Speeds Up Inference

Without KV Cache

```
Token1

↓

Recompute Token1

Token2

↓

Recompute Token1

Recompute Token2

Token3

↓

Recompute Token1

Recompute Token2

Recompute Token3
```

Huge redundant computation occurs.

With KV Cache

```
Token1

↓

Store KV

Token2

↓

Reuse KV(Token1)

Store KV(Token2)

Token3

↓

Reuse KV(Token1)

Reuse KV(Token2)

Store KV(Token3)
```

Inference becomes dramatically faster.

Modern LLMs heavily rely on KV Cache.

---

13. GPT Training

GPT is trained using **Next Token Prediction**.

Example:

Sentence:

```
Deep learning is powerful
```

Training examples:

Input:

```
Deep
```

Target:

```
learning
```

Next,

Input:

```
Deep learning
```

Target:

```
is
```

Next,

Input:

```
Deep learning is
```

Target:

```
powerful
```

Millions of such examples are generated automatically from text corpora.

---

14. Training vs Inference

Training:

- Entire sentence available
- Masking prevents future access
- Parallel computation possible

Inference:

- Future words unknown
- Tokens generated sequentially
- Previous predictions become future inputs

This is why inference is slower than training.

---

15. GPT vs BERT vs Original Transformer

Feature	Original Transformer	BERT	GPT
Encoder	Yes	Yes	No
Decoder	Yes	No	Yes
Bidirectional	Encoder only	Yes	No
Causal Mask	Decoder only	No	Yes
Primary Goal	Translation	Understanding	Generation

---

16. BERT vs GPT

BERT reads text from both directions.

```
Left ← Word → Right
```

This makes BERT excellent for:

- Classification
- Sentiment Analysis
- Named Entity Recognition
- Question Answering

GPT reads only left-to-right.

```
Left → Right
```

This makes GPT ideal for:

- Text Generation
- Chatbots
- Story Writing
- Code Generation
- Summarization

---

17. End-to-End GPT Pipeline

```
Prompt

↓

Tokenizer

↓

Embeddings

↓

Positional Encoding

↓

Masked Multi-Head Attention

↓

Feed Forward Network

↓

Residual Connections

↓

Layer Normalization
```

```
↓

Repeat Decoder Layers

↓

Linear Layer

↓

Softmax

↓

Predict Next Token

↓

Append Token

↓

Repeat
```

This entire pipeline repeats until generation terminates.

---

18. Time Complexity of RNN vs Transformer

Suppose sequence length is

$$n$$

RNN processes sequentially.

Time complexity:

$$O(n)$$

Transformers compute attention between every pair of tokens.

Attention complexity:

$$O(n^2)$$

Although attention is computationally expensive, it is highly parallelizable on GPUs, making Transformers much faster in practice.

---

19. Why Transformers Need So Much Memory

The attention matrix compares every token with every other token.

For

$$n$$

tokens,

matrix size becomes

$$n \times n$$

Memory grows quadratically.

Very long contexts therefore require enormous GPU memory.

This motivates research into efficient attention mechanisms.

---

20. Temperature, Top-k and Top-p Sampling

During generation, GPT does not always choose the highest probability token.

Different decoding strategies exist.

Greedy Decoding

Always choose highest probability.

Produces deterministic output.

Temperature

Controls randomness.

Low temperature:

More deterministic.

High temperature:

More creative and random.

Top-k Sampling

Keep only the top-k most probable tokens.

Sample among them.

Top-p Sampling (Nucleus Sampling)

Choose the smallest set of tokens whose cumulative probability exceeds p.

Then sample from that set.

Modern chatbots often use Top-p sampling.

---

21. Beam Search

Instead of keeping one candidate sentence,

Beam Search keeps multiple candidates.

Example:

Beam Width = 3

The algorithm keeps the three best partial sentences at every step.

Beam Search often produces higher-quality outputs than greedy decoding.

---

22. Evolution of Language Models

```
ANN

↓
```

```
CNN

↓

RNN

↓

LSTM

↓

GRU

↓

Attention

↓

Transformer

↓

BERT / GPT / T5

↓

Large Language Models

↓

ChatGPT
```

Transformers fundamentally changed NLP.

Almost every modern language model uses Transformer architecture.

---

23. Chapter Summary

- GPT stands for Generative Pre-trained Transformer.
- GPT uses only the Transformer Decoder.
- GPT performs next-token prediction.
- Text generation is autoregressive.

- Masking prevents future information leakage.
- KV Cache avoids recomputation during inference.
- Training uses next-token prediction with teacher forcing.
- Inference generates tokens sequentially.
- GPT excels at generation while BERT excels at understanding.
- Modern LLMs are based on Transformer Decoder architectures.

Common Interview Questions

Q1. Why does GPT not need an Encoder?

Because GPT performs only next-token prediction. The prompt itself provides the context.

Q2. What is KV Cache?

A mechanism that stores previously computed Key and Value vectors to avoid recomputation during autoregressive inference.

Q3. Why is masking essential?

It prevents future information leakage and enables autoregressive language modeling.

Q4. Why is inference slower than training?

Because tokens must be generated sequentially during inference.

Q5. Why are Transformers faster than RNNs despite $O(n^2)$ attention complexity?

Because Transformer computations are highly parallelizable on modern GPUs, while RNNs process tokens sequentially.

Final Revision Sheet

Complete Transformer Pipeline

```
Input
```

```
↓

Tokenization

↓

Embeddings

↓

Positional Encoding

↓

Generate Q,K,V

↓

Scaled Dot Product Attention

↓

Multi-Head Attention

↓

Residual + LayerNorm

↓

Feed Forward Network

↓

Residual + LayerNorm

↓

Encoder Output

↓

Decoder

↓

Masked Self-Attention
```

```
↓

Cross Attention (Original Transformer)

↓

Feed Forward

↓

Linear Layer

↓

Softmax

↓

Output Token
```

Key Takeaway

The original Transformer introduced **Attention** as the core computation for sequence modeling.

GPT simplified this architecture by retaining only the **Decoder**, using **causal masking** and **autoregressive next-token prediction**.

Combined with massive pretraining data, efficient GPU computation, and optimizations like **KV Cache**, this architecture evolved into today's **Large Language Models**, capable of reasoning over context, generating coherent text, writing code, answering questions, and powering conversational AI systems such as ChatGPT.